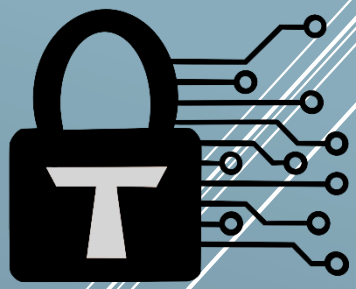
Trust Security



Smart Contract Audit

The Graph – PR #1325

03/05/26

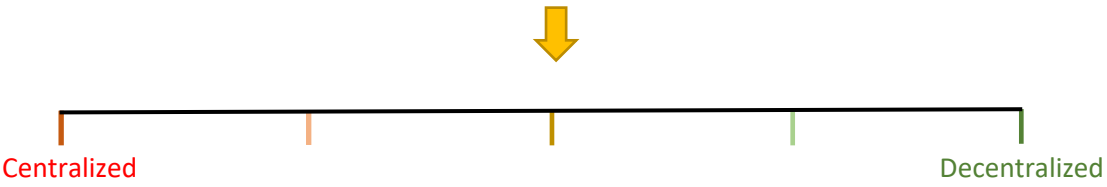
Executive summary

Category	Infrastructure
Auditor	Trust
Time period	03/03/26 – 19/03/26

Findings

Severity	Total	Fixed	Acknowledged
High	4	4	-
Medium	4	3	1
Low	10	9	1

Centralization score



Signature

Digitally signed by Trust
Date: 2026.05.03 17:36:29 +03'00'

Document properties

Versioning

Version	Date	Description
0.1	19/03/26	Client report
0.2	16/04/26	Refactor and fix review
0.3	03/05/26	Fix review

Contact

Trust

trust@trust-security.xyz

Introduction

Trust Security has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

- All non-test files under PR commit

Repository details

- **Repository URL:** https://github.com/graphprotocol/contracts
- **Commit hash:** 7405c9d5f73bce04734efb3f609b76d95ffb520e
- **Fix review commit hash:** 0bbb476f37f85d042927e84d8764fa58eb020ccf
- **2nd Fix review commit hash:** f44fc5a4c74fa5190fd2892ae15a083b79f715f3

About Trust Security

Trust Security has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Since its inception it has safeguarded over 30 clients through private services and over 30 additional projects through bug bounty submissions.

About the Auditors

Trust has established a dominating presence in the smart contract security ecosystem since 2022. He is a resident on the Immunefi, Sherlock and C4 leaderboards and is now focused in auditing and managing audit teams under Trust Security. When taking time off auditing & bug hunting, he enjoys sharing knowledge and experience with aspiring auditors through X or the Trust Security blog.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. Trust Security assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	**Good**	Project kept code as simple as possible, reducing attack risks
Documentation	**Excellent**	Project is extensively documented at both code and off-code level.
Best practices	**Excellent**	Project consistently adheres to industry standards.
Centralization risks	**Good**	Assumed centralization risks are clearly laid out.

Findings

High severity findings

TRST-H-1 Malicious payer gas siphoning via 63/64 rule in collection callbacks leads to collection bypass

- **Category:** Gas-related issues
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

In RecurringCollector._*collect()*, the *beforeCollection()* and *afterCollection()* callbacks to contract payers are wrapped in try/catch blocks (lines 380, 416). A malicious contract payer can exploit the EVM's 63/64 gas forwarding rule to consume nearly all available gas in these callbacks.

The attack works as follows: the malicious payer's *beforeCollection()* implementation consumes 63/64 of the gas forwarded to it, either returning successfully or reverting, but regardless leaving only 1/64 of the original gas for the remainder of _*collect()*. The core payment logic (*PaymentsEscrow.collect()* at line 384) and event emissions then execute with a fraction of the expected gas. The *afterCollection()* callback then consumes another 63/64 of what remains.

Realistically, after both callbacks siphon gas, there will not be enough gas left to complete the *PaymentsEscrow.collect()* call and the subsequent event emissions, causing the entire *collect()* transaction to revert. The security model for Payer as a smart contract does not account for requiring such gas expenditure does not exist, which can also be obfuscated away. This gives the malicious payer effective veto power over all collections against their agreements.

Recommended mitigation

Enforce a minimum gas reservation before each callback. Before calling *beforeCollection()*, check that *gasleft()* is sufficient and forward only a bounded amount of gas using the {gas: **maxCallbackGas**} syntax, retaining enough gas for the core payment logic. Apply the same pattern to *afterCollection()*. This caps the gas available to the payer's callbacks regardless of their implementation, ensuring the critical *PaymentsEscrow.collect()* call always has enough gas to complete.

Team response

Fixed.

Mitigation review

Issue has been fixed as suggested.

TRST-H-2 Invalid supportsInterface() returndata escapes try/catch leading to collection bypass

- **Category:** Logical flaws
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

In RecurringCollector._*collect()* (lines 368-378), the provider eligibility check calls *IERC165(agreement.payer).supportsInterface()* inside a try/catch block. The try clause expects a (bool supported) return value. If the external call succeeds at the EVM level (does not revert) but returns malformed data - such as fewer than 32 bytes of returndata or data that cannot be ABI-decoded as a bool - the Solidity ABI decoder reverts on the caller side when attempting to decode the return value.

This ABI decoding revert occurs in the calling contract's execution context, not in the external call itself. Solidity's try/catch mechanism only catches reverts originating from the external call (callee-side reverts). Caller-side decoding failures escape the catch block and propagate as an unhandled revert, causing the entire _*collect()* transaction to fail.

A malicious contract payer can exploit this by implementing a *supportsInterface()* function that returns success with empty returndata, a single byte, or any non-standard encoding. This permanently blocks all collections against agreements with that payer, since the **code.length** > 0 check always routes through the vulnerable path. As before, the security model does not account for this bypass path to be validated against.

Recommended mitigation

Avoid receiving and decoding values from untrusted contract calls. This can be done manually by reading returndata at the assembly level.

Team response

Fixed.

Mitigation review

Fixed. The affected code has been refactored, addressing the issue.

TRST-H-3 Stale escrow snapshot causes a perpetual revert loop

- **Category:** Logical flaws
- **Source:** RecurringAgreementManager.sol
- **Status:** Fixed

Description

The RecurringAgreementManager (RAM) maintains an **escrowSnap** per *(collector, provider)* pair - a cached view of the escrow balance used to compute **totalEscrowDeficit**. This snap is only updated at the end of _*updateEscrow()* via _*setEscrowSnap()*. When *afterCollection()* is called by the RecurringCollector after a payment collection, the escrow balance has already

been reduced by the collected amount, but **escrowSnap** still reflects the pre-collection value.

The stale-high snap causes *_escrowMinMax()* to understate the **deficit**. In Full escrow mode, when the RAM's free token balance is low, this leads to an incorrect decision to deposit into escrow. The deposit attempt reverts due to insufficient ERC20 balance, and the entire *afterCollection()* call fails. Since RecurringCollector wraps *afterCollection()* in try/catch (line 416), the revert is silently swallowed - but the snap never gets updated, making it permanently stale.

This is self-reinforcing: every subsequent *afterCollection()*, *reconcileAgreement()*, and *reconcileCollectorProvider()* call for the affected pair follows the same code path and reverts for the same reason. There is no manual recovery path. The escrow accounting diverges from reality for the affected pair, and **totalEscrowDeficit** is globally understated, potentially causing other pairs to incorrectly enter Full mode and over-deposit.

The state only self-heals when the RAM receives enough tokens (e.g., from issuance distribution) to cover the phantom deposit, at which point the deposit succeeds but sends tokens to escrow unnecessarily.

Recommended mitigation

Read the fresh escrow balance inside *_escrowMinMax()* when computing the **deficit**, rather than relying on the cached **escrowSnap** derived from **totalEscrowDeficit**. This makes the function self-correcting: even if a prior *afterCollection()* failed, the next call sees the true balance and makes the correct deposit/thaw decision. This approach fixes the root cause rather than masking the symptom with a balance guard.

Team response

Fixed.

Mitigation review

The new code has a *_setEscrowSnap()* call before *_escrowMinMax()*, ensuring the snapshot is updated and fixing the root cause.

TRST-H-4 EOA payer can block collection by acquiring code via EIP-7702

- **Category:** Type confusion
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

In RecurringCollector.*_collect()* (lines 368-378), the provider eligibility gate is applied when **agreement.payer.code.length** > 0. This gate was designed as an opt-in mechanism for contract payers to control which providers can collect. However, with EIP-7702 (live on both Ethereum mainnet and Arbitrum), an EOA can set a code delegation to an arbitrary contract address.

An EOA payer who originally signed an agreement via the ECDSA path can later acquire code using an EIP-7702 delegation transaction. This causes the **code.length** > 0 branch to activate during collection. By delegating to a contract that implements *supportsInterface()* returning true for IProviderEligibility and *isEligible()* returning false, the payer triggers the *require()* on line 373.

The *require()* is inside the try block's success handler. In Solidity, reverts in the success handler are NOT caught by the catch block - they propagate up and revert the entire transaction. This gives the payer complete, toggleable control over whether collections succeed. The payer can enable the delegation to block collections, disable it to sign new agreements, and re-enable it before collection attempts - all at negligible gas cost.

The payer can then thaw and withdraw their escrowed funds after the thawing period, effectively receiving services for free. This bypasses the assumed security model where a provider can trust the escrow balance for an EOA payer to ensure collection will succeed.

Recommended mitigation

Record whether the payer had code at agreement acceptance time by adding a bool flag to the agreement struct (e.g., **payerIsContract**). Only apply the IProviderEligibility gate when the payer was a contract at acceptance. This preserves the eligibility feature for legitimate contract payers while closing the EOA-to-contract vector introduced by EIP-7702.

Team response

Fixed.

Mitigation review

Fixed under the assumption that a provider setting **CONDITION_ELIGIBILITY_CHECK** to true must trust the payer contract. The statement in the fix comment that "*An EOA cannot pass this check, so an EOA cannot create an agreement with eligibility gating enabled*" is inaccurate, because an EOA can always change its code back and forth via EIP-7702 to pass interface checks. The correct security boundary is that the provider trusts the payer contract when opting into eligibility, not that the payer cannot be an EOA.

Medium severity findings

TRST-M-1 Micro-thaw griefing via permissionless depositTo() and reconcileAgreement()

- **Category:** Griefing attacks
- **Source:** RecurringAgreementManager.sol
- **Status:** Fixed

Description

Three independently benign features combine into a griefing vector:

1. *PaymentsEscrow.depositTo()* has no access control - anyone can deposit any amount for any (*payer, collector, receiver*) tuple.
2. *reconcileAgreement()* is permissionless - anyone can trigger a reconciliation which calls *_updateEscrow()*.
3. *PaymentsEscrow.adjustThaw()* with **evenIfTimerReset**=false is a no-op when increasing the thaw amount would reset the thawing timer.

An attacker deposits 1 wei into an escrow account via *depositTo()*, then calls *reconcileAgreement()*. The reconciliation detects escrow is 1 wei above target and initiates a thaw of 1 wei via *adjustThaw()*. This starts the thawing timer. When the RAM later needs to thaw a larger amount (e.g., after an agreement ends or is updated), it calls *adjustThaw()* with **evenIfTimerReset**=false, which becomes a no-op because increasing the thaw would reset the timer.

In cases where thaws are needed to mobilize funds from one escrow pair to another - for example, to fund a new agreement or agreement update for a different provider - this griefing prevents the rebalancing. New agreements or updates that require escrow from the blocked pair's thawed funds could fail to be properly funded, causing escrow mode degradation or preventing the offers entirely.

Recommended mitigation

Add a minimum thaw threshold in *_updateEscrow()*. Amounts below the threshold should be ignored rather than initiating a thaw. This prevents an attacker from starting a thaw timer with a dust amount. If they do perform the attack, they will donate a non-negligible amount in exchange for the one-round block.

Team response

Fixed.

Mitigation review

The griefing path remains reachable. Before any agreement is offered, a 1 wei donation to the *(collector, provider)* escrow account, followed by a permissionless call to *_reconcilePairTracking()* reaches *_updateEscrow()* with min and max at zero, and the thaw threshold is also at zero. Any positive excess passes the **thawThreshold <= excess** check, causing an *adjustThaw(thawTarget = 1)*. The same sequence also occurs after the final collection of an agreement, when **sumMaxNextClaim** transitions to zero via *afterCollection() -> _reconcileAndUpdateEscrow() -> _reconcileAgreement()*. There should be a nominal, non-

negligible minimum thaw amount on top of the fraction check, applied in both *_reconcileProviderEscrow()* and *_withdrawAndRebalance()*. When **escrowBasis** is **JustInTime**, override the nominal skip so that dust can still be thawed out for solvency.

Team Response

The zero-threshold path when sumMaxNextClaim = 0 is acknowledged. Timer resets do not occur (evenIfTimerReset=false rejects increases), so the vector is limited to postponing pair tracking cleanup via repeated dust deposits. Added minResidualEscrowFactor (uint8, default 50, threshold = 2^value ≈ 0.001 GRT for default): pairs with no agreements and escrow below threshold are dropped from tracking. Untracked pairs can still have escrow drained via blind thaw/withdraw on reconcileProvider.

Mitigation review

The main finding is considered acknowledged. Introduction of **minResidualEscrowFactor** is used only in maintaining of view-related bookkeeping. No new concerns have been introduced.

TRST-M-2 The tempJit fallback in beforeCollection() is unreachable in practice

- **Category:** Logical flaw
- **Source:** RecurringAgreementManager.sol
- **Status:** Fixed

Description

In *beforeCollection()* (line 236), when the escrow balance is insufficient for an upcoming collection, the function attempts a JIT (Just-In-Time) top-up by setting **$.tempJit** = true before returning. The **tempJit** flag forces *_escrowMinMax()* to return JustInTime mode, freeing escrow from other pairs to fund this collection.

However, the JIT path is only entered when the escrow is insufficient to cover **tokensToCollect**. In the RecurringCollector.*_collect()* flow, *beforeCollection()* is called before *PaymentsEscrow.collect()*. If *beforeCollection()* cannot top up the escrow (because the RAM lacks free balance and the **deficit** >= *balanceOf()* guard fails), it returns without action. The subsequent *PaymentsEscrow.collect()* then attempts to collect **tokensToCollect** from an escrow that is still insufficient, causing the entire *collect()* transaction to revert.

This means **tempJit** is never set in the scenario where it would be most needed: when escrow is short and the collection will fail regardless. An admin cannot rely on **tempJit** being triggered automatically during the RecurringCollector collection flow and would need to manually set JIT mode to achieve the intended fallback behavior. This would cause a delay the first time the issue is encountered where presumably there is no reason for admin to intervene.

Recommended mitigation

The original intention cannot be trully fulfilled without major redesign of multiple contracts. It is in practice more advisable to take the scenario into account and introduce an off-chain monitoring bot which would set the **tempJit** when needed.

Team response

Fixed.

Mitigation review

The new setup is schematically sound. Admin intervention to trigger **JustInTime** may still be required to satisfy requests when the system is in **OnDemand** but insufficient liquidity is being thawed or minted into the contract.

TRST-M-3 Instant escrow mode degradation from Full to OnDemand via agreement offer

- **Category:** Logical flaw
- **Source:** RecurringAgreementManager.sol
- **Status:** Acknowledged

Description

Neither *offerAgreement()* nor *offerAgreementUpdate()* verify that the RAM has sufficient token balance to fund the new escrow obligation without degrading the escrow mode. An operator can offer an agreement whose **maxNextClaim**, when added to the existing **sumMaxNextClaim**, causes **totalEscrowDeficit** to exceed the RAM's balance. This instantly degrades the escrow mode from Full to OnDemand for ALL *(collector, provider)* pairs.

The degradation occurs because *_escrowMinMax()* checks: **totalEscrowDeficit** < balanceOf(address(this)). When the new agreement pushes the **deficit** above the balance, this condition becomes false, and min drops to 0 for every pair - meaning no proactive deposits are made for any agreement, not just the new one. Existing providers who had fully-escrowed agreements silently lose their escrow guarantees.

Whether intentional or by misfortune, this behavior can be triggered instantly by a single offer. If this degradation is desirable in some cases, it should only occur by explicit intention, not as a side effect of a routine operation.

Recommended mitigation

Add a separate configuration flag (e.g., **allowModeDegradation**) that must be explicitly set by the admin to permit offers that would degrade the escrow mode. When the flag is false, *offerAgreement()* and *offerAgreementUpdate()* should revert if the new obligation would push **totalEscrowDeficit** above the current balance. This ensures mode degradation is always a conscious decision.

Team response

Acknowledged. The risk is documented, including the operator caution about pre-offer headroom checks.

TRST-M-4 Returndata bombing via payer callbacks in _preCollectCallbacks and _postCollectCallback

- **Category:** Gas-related issues
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

All three payer callbacks reachable from *_collect()* (the eligibility **staticcall** in *_preCollectCallbacks()* at line 633, the *beforeCollection()* call in the same function at line 646, and the *afterCollection()* call in *_postCollectCallback()* at line 666) use Solidity's default low-level call pattern, which copies the full returndata buffer into the caller's memory. Note that RETURNDATACOPY is emitted even when the returned bytes are discarded via the **(bool ok,)** tuple pattern.

With a forwarded budget of **MAX_PAYER_CALLBACK_GAS** (1,500,000) per callback, a malicious payer can expand callee memory and return roughly 850 KB of data. The caller's RETURNDATACOPY and the associated memory expansion then consume approximately 1,500,000 gas in the *_collect()* frame for each callback. Across the three callbacks, a single *collect()* call can be forced to burn about 4,500,000 gas beyond the nominal callback budget.

The impact is an inflated collection cost that is not reflected in off-chain gas estimates. This is gas griefing rather than a collection block, and gas costs remain manageable.

Recommended mitigation

Replace the affected high-level call sites with inline assembly that performs the call and bounds the amount of returndata copied. For the eligibility check, copy at most 32 bytes into scratch memory and read the result. For *beforeCollection()* and *afterCollection()*, copy zero bytes since the return value is unused.

Team response

Replaced all three call sites with inline assembly that bounds returndata copy:

- **Eligibility staticcall**: copies at most 32 bytes into scratch space (0x00), reads the uint256 result from there.
- **beforeCollection / afterCollection**: copy 0 bytes (retSize=0), only the bool success from the CALL opcode is used.

This prevents a malicious payer from forcing RETURNDATACOPY of ~850 KB per callback.

Mitigation review

Issue has been addressed as suggested.

Low severity findings

TRST-L-1 Insufficient gas for afterCollection callback leaves escrow state outdated

- **Category:** Time sensitivity flaw
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

In RecurringCollector._*collect()*, after a successful escrow collection, the function notifies contract payers via a try/catch call to *afterCollection()* (line 416). The caller (originating at data provider) controls the gas forwarded to the *collect()* transaction. By providing just enough gas for the core collection to succeed but not enough for the *afterCollection()* callback, the external call will revert due to an out-of-gas error, which is silently caught by the catch block.

For the RecurringAgreementManager (RAM), *afterCollection()* triggers *_reconcileAndUpdateEscrow()*, which reconciles the agreement's **maxNextClaim** against on-chain state and updates the escrow snapshot via *_setEscrowSnap()*. When this callback is skipped, the **escrowSnap** remains at its pre-collection value, overstating the actual escrow balance. This stale snapshot causes **totalEscrowDeficit** to be understated, which can lead to incorrect escrow mode decisions in *_escrowMinMax()* for subsequent operations on the affected *(collector, provider)* pair.

The state will self-correct on the next successful call to *_updateEscrow()* for the same pair (e.g., via *reconcileAgreement()* or a subsequent collection with sufficient gas), so the impact is temporary. However, during the stale window, escrow rebalancing decisions may be suboptimal.

Recommended mitigation

Enforce a minimum gas forwarding requirement for the *afterCollection()* callback. This can be done by checking *gasleft()* before the *afterCollection()* call and reverting if insufficient gas remains for the callback to execute meaningfully.

Team response

Fixed.

Mitigation review

Fixed as suggested.

TRST-L-2 Pending update over-reserves escrow with unrealistically conservative calculation

- **Category:** Arithmetic issues
- **Source:** RecurringAgreementManager.sol
- **Status:** Fixed

Description

In *offerAgreementUpdate()* (line 328), the pending update's **maxNextClaim** is computed via *_computeMaxFirstClaim()* using the full **maxSecondsPerCollection** window and the new **maxInitialTokens**. This amount is added to **sumMaxNextClaim** alongside the existing (non-pending) **maxNextClaim**, making both slots additive.

This is overly conservative because only one set of terms is ever active at a time. While the update is pending, the RAM reserves escrow for both the current agreement terms and the proposed updated terms simultaneously. The correct calculation should take the maximum of the two rates multiplied by **maxSecondsPerCollection** plus the new **maxInitialTokens**, and add the old **maxInitialTokens** only if the initial collection has not yet occurred.

The over-reservation reduces the effective capacity of the RAM, ties up capital that could serve other agreements, and in Full mode can trigger escrow mode degradation by inflating **totalEscrowDeficit**. Once the update is accepted or revoked, the excess is released, but during the pending window the impact on escrow accounting is significant for high-value agreements. Additionally, the over-reservation will trigger an unnecessary thaw as soon as the agreement update completes, since escrow will exceed the corrected target.

Recommended mitigation

The **pendingMaxNextClaim** should be computed as stated above, then reduced by the current **maxNextClaim** so that the total deficit is accurate. This reflects the reality that only one set of terms is active at any time, and the worst-case scenario where *collect()* is called before and after the agreement update.

Team response

Fixed.

Mitigation review

Refactored so that at any point, the accurate worst-case collection is reflected.

TRST-L-3 Unsafe behavior of approveAgreement during pause

- **Category:** Access control issues
- **Source:** RecurringAgreementManager.sol
- **Status:** Fixed

Description

The *approveAgreement()* function (line 226) is a view function with no **whenNotPaused** modifier. During a pause, it continues to return the magic selector for authorized hashes, allowing the RecurringCollector to accept new agreements or apply updates even while the RAM is paused.

A pause is typically an emergency measure intended to halt all state-changing operations. Allowing agreement acceptance during pause undermines this intent, as the accepted agreement creates obligations (escrow reservations, **maxNextClaim** tracking) that the paused RAM cannot manage.

Similarly, *beforeCollection()* and *afterCollection()* do not check pause state. While blocking these during pause could prevent providers from collecting earned payments, allowing them could pose a security risk if the pause was triggered due to a discovered vulnerability in the escrow management logic.

Recommended mitigation

Add a pause check to *approveAgreement()* that returns *bytes4(0)* when the contract is paused, preventing new agreement acceptances and updates during emergency pauses. For *beforeCollection()* and *afterCollection()*, evaluate the trade-off: blocking them protects against exploitation of escrow logic bugs during pause, while allowing them ensures providers can still collect earned payments. Consider allowing collection callbacks only in a restricted mode during pause.

Team response

Fixed.

Mitigation review

Fixed. Underlying code has been refactored, addressing the issue.

TRST-L-4 Pair tracking removal blocked by 1 wei escrow donation

- **Category:** Donation attacks
- **Source:** RecurringAgreementManager.sol
- **Status:** Acknowledged

Description

When the last agreement for a *(collector, provider)* pair is deleted, *_reconcilePairTracking()* is intended to remove the pair from the tracking sets (**collectorProviders**, **collectors**) and clean up the escrow state. However, an attacker can prevent this cleanup by depositing 1 wei of GRT into the pair's escrow account via *PaymentsEscrow.deposit()* just before the reconciliation occurs.

The donation increases the escrow balance, which in turn updates the **escrowSnap** to a non-zero value during *_updateEscrow()*. The *_reconcilePairTracking()* function checks whether the **escrowSnap** is zero to determine if the pair can be safely removed. With the 1 wei donation, this check passes (snap != 0), and the pair is retained in the tracking sets even though it has no active agreements.

This leaves orphaned entries in the **collectorProviders** and **collectors** tracking sets, preventing clean removal of the collector from the RAM's accounting.

Recommended mitigation

In *_reconcilePairTracking()*, base the removal decision on **pairAgreementCount** reaching zero rather than on **escrowSnap** being zero. If no agreements remain for a pair, remove it from

tracking regardless of the escrow balance. Any residual escrow balance (from donations or rounding) can be handled by initiating a thaw before removal.

Team response

Accepted limitation. Orphaned tracking entries do not affect correctness or funds safety.

TRST-L-5 The _computeMaxFirstClaim function overestimates when deadline is before full collection window

- **Category:** Logical flaw
- **Source:** RecurringAgreementManager.sol
- **Status:** Fixed

Description

In *_computeMaxFirstClaim()* (line 645), the maximum first claim is computed as: **maxOngoingTokensPerSecond** * **maxSecondsPerCollection** + **maxInitialTokens**. This uses the full **maxSecondsPerCollection** window regardless of how much time actually remains until the agreement's **endsAt** deadline.

In contrast, RecurringCollector's *getMaxNextClaim()* correctly accounts for the remaining time until the deadline, capping the collection window when the deadline is closer than **maxSecondsPerCollection**. The RAM's overestimate means **sumMaxNextClaim** is inflated for agreements near their end date, causing the RAM to reserve more escrow than the RecurringCollector would ever allow to be collected.

The excess reservation is wasteful but not directly exploitable, as the collector enforces the actual cap during collection. However, it reduces the RAM's effective capacity and can contribute to unnecessary escrow mode degradation.

Recommended mitigation

Align *_computeMaxFirstClaim()* with the RecurringCollector's *getMaxNextClaim()* logic by accounting for the remaining time until the agreement's **endsAt**. Compute the collection window as min(**maxSecondsPerCollection**, **endsAt** - **lastCollectionAt**) when determining the maximum possible claim. This requires passing the **endsAt** parameter to the function.

Team response

Fixed.

Mitigation review

Fixed. The RecurringCollector now calculates the effective window correctly.

TRST-L-6 The cancel() function order sensitivity leaves RCAU offer unreachable

- **Category:** Time-sensitivity issues
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

When a payer has both a pending RCA offer and a pending RCAU offer for the same **agreementId** and neither has been accepted, the order of cancellations matters. The *cancel()* overload that takes a terms hash delegates authorization to *_requirePayer()* (lines 480-497), which first checks the accepted agreement's **payer** and then the stored **rcaOffers** entry's payer. It does not fall back to **rcauOffers**.

If the payer first cancels the RCA offer under **SCOPE_PENDING**, the entry in **rcaOffers** is deleted. A subsequent attempt to cancel the RCAU offer then fails: *_requirePayer()* finds no accepted agreement and no RCA offer, and reverts with **RecurringCollectorAgreementNotFound**. The orphaned RCAU offer remains in storage and unreachable by the payer. If the same parameters are later re-used to offer a new RCA, the orphaned RCAU is associated with it. The **updateNonce** check prevents immediate acceptance of the stale RCAU, but the payer has lost the ability to clean up state they own.

Recommended mitigation

Extend *_requirePayer()* to also check **rcauOffers** for a payer match when neither an accepted agreement nor an RCA offer is present. Alternatively, enforce symmetric cleanup so that deleting an RCA offer under **SCOPE_PENDING** also deletes any **rcauOffers** entry with the same **agreementId**.

Team response

Resolved by persisting agreement.payer from the first offer() instead of waiting until accept(). _requirePayer is replaced by an inline agreement.payer check at the cancel() call site, reading the persisted address directly without falling back through rcaOffers. _offerUpdate likewise reads agreement.payer instead of decoding the stored RCA bytes on every update offer. As a consequence, cancelling a pre-acceptance RCA offer and cancelling a pending RCAU offer are fully independent operations that may be performed in either order — neither path leaves the other unreachable, because the persistent agreement.payer continues to authorize the surviving offer.

Mitigation review

The restructuring of agreement data ensures no offer is orphaned and unreachable.

TRST-L-7 EOA payer signatures cannot be revoked before deadline

- **Category:** Functionality flaws
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

Payers approve agreements through two paths: an ECDSA signature consumed by *accept()* or *update()*, and a stored offer placed by a contract payer via *offer()* and consumed against the stored hash. Contract payers can revoke a pending offer by calling *cancel()* with **SCOPE_PENDING**, which deletes the matching entry from **rcaOffers** or **rcauOffers**.

EOA payers have no equivalent revocation path. Once an RCA or RCAU has been signed, the signature is accepted by the collector at any time before the **deadline** field expires. A payer that wishes to cancel a signature-based offer before the deadline (for example, to renegotiate terms) has no mechanism to do so. The only remaining option to ensure no duplicate agreement risk is to wait out the deadline (and hope their unintended offer is not matched), or to revoke the signer via the **Authorizable** thawing and revocation flow, which affects all agreements authorized by that signer rather than an individual offer.

Recommended mitigation

Expose a *cancelSignature(bytes32 hash)* entry point that records the hash as invalidated on-chain, and have *_requireAuthorization()* reject any hash that has been invalidated. Alternatively, use a per-signer nonce that the payer can bump to invalidate all outstanding signatures for that signer.

Team response

Added SCOPE_SIGNED flag to cancel(), giving EOA signers an on-chain revocation path like contract payers already have via SCOPE_PENDING. The signer calls cancel(agreementId, termsHash, SCOPE_SIGNED) which records cancelledOffers[msg.sender][termsHash] = agreementId. When accept() or update() later processes a signature, _requireAuthorization recovers the signer via ECDSA and rejects if the stored agreementId matches. Self-authenticating (keyed by signer address), idempotent, reversible (calling again with bytes16(0) undoes the cancellation), and combinable with other scopes. Also made cancel no-op when nothing exists on-chain instead of reverting.

Mitigation review

The introduced alternative cancel path is sufficient. It should be clarified that whenever a payer is represented by a signer, *cancel()* should be called by the signer, not payer.

TRST-L-8 Callback gas precheck does not account for intermediate overhead

- **Category:** Gas-related issues
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

Both *_preCollectCallbacks()* and *_postCollectCallback()* guard each payer callback with a precheck of the form **if (gasleft() < (MAX_PAYER_CALLBACK_GAS * 64) / 63) revert**. The intent is to ensure that **MAX_PAYER_CALLBACK_GAS** remains available to the callee after applying the EIP-150 63/64 rule.

However, the precheck is performed before the CALL or STATICCALL opcode itself, and additional gas is consumed between the comparison and the opcode: local Solidity operations, stack and memory setup, calldata encoding, and the fixed cost of the CALL or STATICCALL

instruction. The actual gas forwarded to the callee can fall below **MAX_PAYER_CALLBACK_GAS**. An honest callee may perform incorrect logic under the assumption of available gas. One can refer to Optimism's **CrossDomainMessenger**, which adds explicit buffer constants (**RELAY_GAS_CHECK_BUFFER** and **RELAY_CALL_OVERHEAD**) for this exact reason.

Recommended mitigation

Add explicit buffer constants to the precheck so that the comparison accounts for the CALL/STATICCALL cost and the intervening Solidity overhead. Size the buffer so that at least **MAX_PAYER_CALLBACK_GAS** is forwarded to the callee when the check passes.

Team response

Added CALLBACK_GAS_OVERHEAD = 3_000 constant. All three prechecks now use:

```
if (gasleft() < (MAX_PAYER_CALLBACK_GAS * 64) / 63 + CALLBACK_GAS_OVERHEAD)
    revert RecurringCollectorInsufficientCallbackGas();
```

Sized to cover the worst-case pre-opcode cost. The eligibility STATICCALL is the first access to the payer account on the collect path, so the EIP-2929 cold-account access cost (2_600) dominates; the remaining headroom covers abi.encodeCall and stack/memory setup. Subsequent beforeCollection / afterCollection calls hit the payer warm (100 gas access), so the buffer is generous there.

Follows the Optimism buffer-constant pattern as suggested.

Mitigation review

Issue has been addressed as suggested. Worst-case scenario tests should be introduced to ensure the defined overhead constant is sufficient in future builds.

TRST-L-9 EIP-7702 payer code change enables callback gas griefing after acceptance

- **Category:** Type confusion
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

Under EIP-7702, which is live on Ethereum mainnet and Arbitrum, an EOA can install arbitrary code via a delegation transaction. *_preCollectCallbacks()* and *_postCollectCallback()* dispatch the *beforeCollection()* and *afterCollection()* callbacks only when **payer.code.length != 0**. A payer who accepted an agreement as an EOA can later acquire code, and have the callbacks dispatched against delegated code that the service provider never considered at acceptance time.

The callbacks are low level calls with a **MAX_PAYER_CALLBACK_GAS** budget, and they are vulnerable to the returndata bombing vector described in TRST-M-4, on top of the baseline call costs. Service providers estimate gas for *collect()* under the assumption that the payer is an EOA with no callbacks. If the payer is a contract at collection time, the provider's gas estimate may be insufficient and the transaction will revert with griefed gas. This is a distinct attack surface from TRST-H-4, which targeted the eligibility gate rather than the callback path.

Recommended mitigation

Use the introduced **CONDITION_ELIGIBILITY_CHECK** flag in place of the live **code.length** check in *_preCollectCallbacks()* and *_postCollectCallback()*. This freezes the contract-versus-EOA determination to the state the service provider observed at acceptance.

Team response

Reusing CONDITION_ELIGIBILITY_CHECK for callback dispatch avoided because the eligibility checking is a different concern with different trust assumptions. An agreement can legitimately have one without the other.

Introduced CONDITION_AGREEMENT_OWNER flag that mirrors the eligibility pattern:

- _requirePayerInterfaceSupport validates IERC165(payer).supportsInterface(type(IAgreementOwner).interfaceId) if the flag is set, alongside the existing eligibility check.
- _preCollectCallbacks and _postCollectCallback dispatch on agreement.conditions & CONDITION_AGREEMENT_OWNER, replacing the payer.code.length check.

Mitigation review

The root cause has been addressed. It is recommended to document that turning CONDITION_ELIGIBILITY_CHECK on an EOA is considered fully trusting it as it can upgrade to a contract that reverts the eligibility check.

TRST-L-10 Inaccurate state flags returned by getAgreementDetails() and _offerUpdate()

- **Category:** Logical flaws
- **Source:** RecurringCollector.sol
- **Status:** Fixed

Description

The **IAgreementCollector** interface defines state bit flags including **ACCEPTED** and **UPDATE**, with the documented convention that **UPDATE** is ORed into the state returned by *getAgreementDetails()* for pending versions (index 1). Two deviations from the specification were observed.

First, in *_offerUpdate()* (lines 417 to 455), when an update is offered against an already accepted agreement, the returned **AgreementDetails** sets **state** to **REGISTERED | UPDATE** without ORing **ACCEPTED**. Callers that inspect the returned state to determine whether the agreement is already live will misread the underlying agreement as not accepted.

Second, in *getAgreementDetails()* (lines 500 to 528), the **UPDATE** bit is never ORed into the returned state for the pending version path. The interface documentation promises this behavior for pending versions, but the implementation returns **REGISTERED** or **ACCEPTED** without regard to whether an RCAU offer is pending.

Neither deviation changes on-chain accounting, but integrators relying on the declared state semantics will receive misleading data.

Recommended mitigation

In *_offerUpdate()*, OR the **ACCEPTED** bit into **state** when the underlying agreement is in the **Accepted** state. In *getAgreementDetails()*, OR the **UPDATE** bit into the returned state when a pending RCAU offer exists for the agreement.

Team response

getAgreementDetails() previously ignored the index parameter and returned only ACCEPTED for any agreement past NotAccepted, regardless of whether a pending RCAU also existed. It now honors index as a generic version selector with two named aliases:

- VERSION_CURRENT = 0 — the active version. For an accepted agreement, returns agreement fields + activeTermsHash with REGISTERED | ACCEPTED, plus UPDATE when the active terms came from an update. Pre-acceptance, returns the stored RCA offer with REGISTERED. Identity (payer, dataService, serviceProvider) is read from agreement storage in both cases; these fields are now persisted in offer().
- VERSION_NEXT = 1 — the next queued version: a pending RCAU awaiting acceptance. Returns REGISTERED | UPDATE when present; empty once accepted (at which point it has moved to VERSION_CURRENT).

getAgreementOfferAt() mirrors the same per-version semantics: VERSION_CURRENT returns the offer that produced activeTermsHash (RCA pre-update or RCAU post-update); VERSION_NEXT returns the pending RCAU when distinct from the active hash.

offer() and getAgreementDetails() share a state composer keyed by version index, so both surfaces report identical flags. Flags split into per-version (REGISTERED, ACCEPTED, UPDATE, SETTLED) and per-agreement (NOTICE_GIVEN, BY_PAYER, BY_PROVIDER) groups. ACCEPTED is set only when the queried version equals activeTermsHash; SETTLED is scoped to the version's own claim (active or pending) so a non-zero claim on one version does not suppress SETTLED on the other.

After update() promotes an RCAU to active, those bytes live in the RCAU slot. A subsequent offer(OFFER_TYPE_UPDATE) with a different hash overwrites that slot and the active RCAU's bytes, therefore they cannot be returned by getAgreementOfferAt(id, VERSION_CURRENT). Resolving this without a hash-keyed terms store would require a third storage slot or a flexible-type slot, both judged disproportionate to the observability concern.

Mitigation review

The latest changes implement a more consistent viewing interface for the contracts, addressing the issues identified.

Additional recommendations

TRST-R-1 Avoid redeployment of the RewardsEligibilityOracle by restructuring storage

The modified RewardsEligibilityOracle has two new state variables, as well as moving **eligibilityValidationEnabled** from the original slot to the end of the structure. Due to the relocation, an upgrade is needed, meaning all previous eligibility state will be lost. It is possible to only append storage slots to the original structure, and avoid a hard redeployment flow, by leveraging the upgradeability of the oracle.

TRST-R-2 Improve stale documentation

The functions below are mentioned in various documentation files but do not exist in the current codebase:

- *acceptUnsignedIndexingAgreement()*
- *removeAgreement()*

TRST-R-3 Incorporate defensive coding best practices

In the RAM's *cancelAgreement()* function, the agreement state is required to not be not accepted. However, the logic could be more specific and require the agreement to be Accepted - rejecting previously cancelled agreements. There is no impact because corresponding checks in the RecurringCollector would deny such cancels, but it remains as a best practice.

TRST-R-4 Document critical assumptions in the RAM

The *approveAgreement()* view checks if the agreement hash is valid, however it offers no replay protection for repeated agreement approvals. This attack vector is only stopped at the RecurringCollector as it checks the agreement does not exist and maintains unidirectional transitions from the agreement Accepted state. For future collectors this may not be the case, necessitating clear documentation of the assumption.

TRST-R-5 Ambiguous return value in getAgreementOfferAt()

getAgreementOfferAt() returns **(uint8 offerType, bytes memory offerData)**. The offer type constant **OFFER_TYPE_NEW** is defined as **0**, which is also the default Solidity return value when no stored offer exists for the given **agreementId** and **index**. A caller receiving **offerType == 0** cannot distinguish between a stored new-type offer existing and no offer existing. Consider redefining offer type constants with 1-indexed values, or adding an explicit **bool found** return parameter.

TRST-R-6 Dead code guard in _validateAndStoreUpdate()

In *_validateAndStoreUpdate()* (line 855), the guard **if (oldHash != bytes32(0))** is unreachable as a false branch. Only agreements in the **Accepted** state may be updated, and every accepted agreement has a non-zero **activeTermsHash** written during *accept()* or a prior *update()*. The guard can be removed or converted into an invariant comment documenting this assumption.

TRST-R-7 Remove consumed offers in accept() and update()

After *accept()* or *update()* consumes a stored offer, the corresponding entry in **rcaOffers** or **rcauOffers** becomes stale. Currently only *_validateAndStoreUpdate()* cleans up the previously active offer by looking up the old **activeTermsHash**; the offer whose terms were just accepted is not deleted. This is a storage hygiene concern: stale offer entries remain in storage indefinitely until explicitly replaced or matched by a future update. Consider deleting the consumed offer entry inside *accept()* and *update()* after it has been applied.

TRST-R-8 Align pause documentation with callback behavior in the RAM

The **RecurringAgreementManager** documentation header states that pausing the contract "*stops all permissionless escrow management*". In practice, the **whenNotPaused** modifier also applies to *beforeCollection()* and *afterCollection()*, so pause also halts the callback path used during *collect()*. Update the documentation to reflect that callbacks are affected, or narrow the modifier application so that behavior matches the prose.

TRST-R-9 _isAuthorized() override in RecurringCollector trusts itself for any authorizer

The *_isAuthorized(address authorizer, address signer)* override in **RecurringCollector** returns true whenever **signer == address(this)**, regardless of **authorizer**. This enables **RecurringCollector** to call *dataService.cancelIndexingAgreementByPayer()* on the payer's behalf. The semantics are safe in the current integration with **SubgraphService**, but they widen the trust surface: any future consumer that relies on *RecurringCollector.isAuthorized()*

for access control will grant access when the signer is the collector itself. Consider tightening the override to scope trust to specific callers, or explicitly document the integration contract so it is not misapplied by future consumers.

TRST-R-10 Document role-change semantics for existing agreements

Changes to **DATA_SERVICE_ROLE** and **COLLECTOR_ROLE** on the **RecurringAgreementManager** do not affect agreements that have already been offered or accepted through the previously authorized addresses. This is by design (revoking a role should not invalidate settled obligations), but the behavior is not documented. Record this invariant in the RAM documentation so that operators and integrators understand the effect of role changes.

TRST-R-11 Remove or implement unused state flags in IAgreementCollector

IAgreementCollector defines state flag constants that are not currently used in the **RecurringCollector** implementation, including **NOTICE_GIVEN**, **SETTLED**, **BY_PAYER**, **BY_PROVIDER**, **BY_DATA_SERVICE**, **AUTO_UPDATE**, and **AUTO_UPDATED**. Unused public interface constants are a source of confusion for integrators, who may code against documented semantics that the implementation does not honor. Either remove the unused flags from the interface, or implement the behaviors they describe in the collector.

TRST-R-12 Document ACCEPTED state returned for cancelled agreements

In *getAgreementDetails()*, any agreement whose state is not **AgreementState.NotAccepted** is reported with state flag **ACCEPTED**. This includes agreements that have been cancelled (**CanceledByPayer** or **CanceledByServiceProvider**). Integrators inspecting the returned state cannot distinguish cancelled agreements from live ones without reading separate storage. Document this behavior in the interface, or extend the state bitmask with a **CANCELED** flag and return it for the non-active terminal states.

TRST-R-13 Document reclaim reason change for stale allocation force-close

Before the PR's refactor, *forceCloseStaleAllocation()* closed the allocation via *_closeAllocation()* and caused a reclaim with reason **CLOSE_ALLOCATION**. Post refactor, the force close path goes through *_resizeAllocation(allocationId, 0, ...)*, which triggers a reclaim with reason **STALE_POI** instead. The reclaim still occurs, but the reason code exposed to reclaim address configuration changes. Document this change so that operators are able to prepare accordingly and have funding paths line up with intention.

TRST-R-14 Avoid magic numbers in production code

In the RecurringAgreementManager, *_getAgreementProvider()* calls *getAgreementDetails()* passing 0 as index. While in previous this was not used on the RecurringCollector, it is now handled as VERSION_CURRENT for correct logic of the collector. Consider using the constant from IAgreementCollector for futureproofing of the contract and clarifying the intention.

Centralization risks

TRST-CR-1 RAM Governor has unilateral control over payment infrastructure

The RecurringAgreementManager's **GOVERNOR_ROLE** has broad unilateral authority over critical payment infrastructure:

- Controls which data services can participate (**DATA_SERVICE_ROLE** grants)
- Controls which **collectors** are trusted (**COLLECTOR_ROLE** grants)
- Can set the issuance allocator address, redirecting the token flow that funds all escrow
- Can set the provider eligibility oracle, which gates who can receive payments
- Can pause the entire contract, halting all agreement management

A compromised or malicious governor could revoke a data service's role (preventing new agreements), change the issuance allocator to a contract that withholds funds, or set a malicious eligibility oracle that blocks specific providers from collecting. These actions affect all agreements managed by the RAM, not just future ones.

TRST-CR-2 Operator role controls agreement lifecycle and escrow mode

The **OPERATOR_ROLE** (admin of **AGREEMENT_MANAGER_ROLE**) controls the operational layer of the RAM:

- Grants **AGREEMENT_MANAGER_ROLE**, which authorizes offering, updating, revoking, and canceling agreements
- Can change the **escrowBasis** (Full/OnDemand/JIT), instantly affecting escrow behavior for all existing agreements
- Can set **tempJit**, overriding the escrow mode to JIT for all pairs

An operator switching from Full to JIT mode instantly removes proactive escrow guarantees for all providers. Providers who accepted agreements under the assumption of Full escrow backing may find their payment security degraded without notice or consent. The escrow mode change is a storage write with no timelock or multi-sig requirement.

TRST-CR-3 Single RAM instance manages all agreement escrow

The RecurringAgreementManager is a single contract instance that manages escrow for all agreements across all (collector, provider) pairs. The **totalEscrowDeficit** is a global aggregate, and the escrow mode (Full/OnDemand/JIT) applies uniformly to all pairs.

This means operational decisions or issues affecting one pair can cascade to all others. For example, a single large agreement that becomes insolvent increases **totalEscrowDeficit**,

potentially degrading the escrow mode from Full to OnDemand for every other pair. Similarly, a stale snapshot on one pair (TRST-H-3) affects the global **deficit** calculation.

There is no isolation between pairs beyond the per-pair **sumMaxNextClaim** tracking. The RAM does not support per-pair escrow mode configuration or per-pair balance ringfencing.

Systemic risks

TRST-SR-1 JIT mode provider payment race condition

When the RecurringAgreementManager operates in JustInTime (JIT) escrow mode, escrow is not proactively funded for any (collector, provider) pair. Instead, funds are deposited into escrow only during the *beforeCollection()* callback, moments before *PaymentsEscrow.collect()* executes. Since the RAM holds a shared pool of GRT that backs all agreements, multiple providers collecting around the same time are effectively racing for the same pool of tokens.

If the RAM's balance is sufficient to cover any single collection but not all concurrent collections, the provider whose data service submits the *collect()* transaction first will succeed, while subsequent providers' collections will revert because the RAM's balance has been depleted by the first collection's JIT deposit. This creates a first-come-first-served dynamic where providers must compete on transaction ordering to receive payment.

This race condition is inherent to the JIT mode design and cannot be fully eliminated without proactive escrow funding. In extreme cases, a well-resourced provider could use priority gas auctions or private mempools to consistently front-run other providers' collections, creating an unfair payment advantage unrelated to service quality.

TRST-SR-2 Escrow thawing period creates prolonged fund immobility

The PaymentsEscrow thawing period (configurable up to **MAX_WAIT_PERIOD**, 90 days) creates a window during which escrowed funds are immobile. When the RAM needs to rebalance escrow across providers - for example, after an agreement ends and funds should be redirected to a new agreement - the thawing delay prevents immediate reallocation. During this window, the RAM effectively has reduced capacity.

If multiple agreements end in a short period or the escrow mode degrades from Full to OnDemand, the RAM may enter a state where substantial funds are locked in thawing and unavailable for either existing or new obligations. This is compounded by the micro-thaw griefing vector (TRST-M-1), which can extend the immobility period by blocking thaw increases.

The thawing period is a protocol-level parameter set on PaymentsEscrow and is outside the RAM's control. Changes to this parameter affect all users of the escrow system, not just the RAM.

TRST-SR-3 Issuance distribution dependency for RAM solvency

The RAM relies on periodic issuance distribution (via the issuance allocator) to receive GRT tokens for funding escrow obligations. If the issuance system experiences delays, governance

disputes, or contract upgrades that temporarily halt distributions, the RAM's free balance depletes as collections drain escrow without replenishment.

Once the free balance reaches zero, the RAM cannot fund JIT top-ups in *beforeCollection()*, cannot proactively deposit in Full mode for new agreements, and existing escrow accounts gradually drain with each collection. Prolonged issuance interruption could cascade into escrow mode degradation (Full → OnDemand → JIT), ultimately affecting all providers' payment reliability.

This is an external dependency that the RAM admin cannot mitigate beyond maintaining a buffer balance.

TRST-SR-4 Try/catch callback pattern silently degrades state consistency

The RecurringCollector wraps all payer callbacks (*beforeCollection()*, *afterCollection()*) in try/catch blocks. While this design prevents malicious or buggy payer contracts from blocking collection, it means that any revert in these callbacks is silently discarded. The collection proceeds as if the callback succeeded, but the payer's internal state (escrow snapshots, **deficit** tracking, reconciliation) may not have been updated.

This creates a systemic tension: the try/catch is necessary for liveness (ensuring providers can collect), but it trades state consistency for availability. Over time, if callbacks fail repeatedly (due to gas issues, contract bugs, or the stale snapshot issue in TRST-H-3), the divergence between the RAM's internal accounting and the actual escrow state can compound silently with no on-chain signal.

There is no event emitted when a callback fails, making it difficult for off-chain monitoring to detect and respond to these silent failures.